Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 3

Wildcards, Hard Links and Symbolic Links

Lab 3 guide: select files with patterns, then see what a rename does to a hard link and to a symbolic link.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Explain which filenames a Bash wildcard selects, and what quoting changes.
2. Use inode numbers to tell a hard link from a separate copy.
3. Explain and repair a symbolic link after its target is renamed.

THIS LAB

Lab 3

First the concepts, with examples. Then the lab on the server. GRUB and shared libraries are optional extensions, so they come at the end.

DISCUSSION

You rename a report. What breaks?

A shortcut on the desktop, a second name for the same file, a note that says where the file is. Which of them still work?

CONCEPTS FIRST

Wildcards and links

Two ideas: the shell turns patterns into names, and one file can have more than one name.

A wildcard is a pattern, and the shell expands it

- A wildcard is a character that stands for other characters, such as * for any characters.
- The shell, not the command, expands the pattern into a list of matching names, then runs the command with that list.
- If nothing matches, Bash keeps the pattern as typed. The command then looks for a file with that exact name.
- Wildcards select file names. They are not the same as regular expressions.

BEFORE THE LAB

Why learn it here?

One pattern replaces a long list of names. The same pattern, used carelessly with rm, removes more than you meant.

The wildcard patterns

Pattern	Meaning	Example	Matches
`*`	Zero or more characters	`*.txt`	every .txt file
`?`	Exactly one character	`file?.txt`	file1.txt, fileA.txt
`[abc]`	One character from the set	`file[123].txt`	file1.txt, file2.txt, file3.txt
`[a-z]`	One character in the range	`[A-Z]*.txt`	names that start with a capital
`[!abc]`	One character not in the set	`file[!0-9].txt`	fileA.txt, not file1.txt
`{a,b}`	Brace expansion: generates words	`file.{txt,log}`	the words file.txt and file.log

Square brackets match one character that exists. Braces only write words, before any file is looked at.

Patterns at work

student@server: ~/oslab-work/lab3

```
$ ls *.txt
$ ls report0[1-3].txt
$ ls log_[!j]*.log
$ touch memo_{mon,tue,wed}.txt
$ cp *.csv archive/
$ ls *.tmp
$ rm *.tmp
```

The first three list files. touch with braces creates three files in one command.

cp copies every .csv file into archive/.

CAUTION

Preview before you delete

Run ls *.tmp first. If the list is what you expect, then run rm *.tmp.

See what the shell really passes

student@server: ~/oslab-work/lab3

```
$ mkdir -p practice
$ printf 'one\n' > practice/report1.txt
$ printf 'two\n' > practice/report2.txt
$ printf 'log\n' > practice/report.log

$ printf '<%s>\n' practice/*.txt
<practice/report1.txt>
<practice/report2.txt>
$ printf '<%s>\n' 'practice/*.txt'
<practice/*.txt>
```

printf prints each word it receives between angle brackets, one per line.

Without quotes, the shell sends two words: the two .txt files. With single quotes around the whole pattern, it sends one word with a literal *.

Use this trick to check a pattern before you give it to cp, mv or rm.

Brackets and braces are not the same

Square brackets look at files

```
student@server: ~/oslab-work/lab3

$ ls practice/report[12].txt
practice/report1.txt  practice/report2.txt
$ ls practice/report[13].txt
ls: cannot access 'practice/report3.txt': No such file
```

Only names that exist can match. Bash keeps a pattern with no match, so the error names the pattern.

Braces only write words

```
student@server: ~/oslab-work/lab3

$ echo practice/report{1,3}.txt
practice/report1.txt practice/report3.txt
```

The file report3.txt does not exist, and echo does not care. Brace expansion happens first, and no file is looked at.

A name is not the file

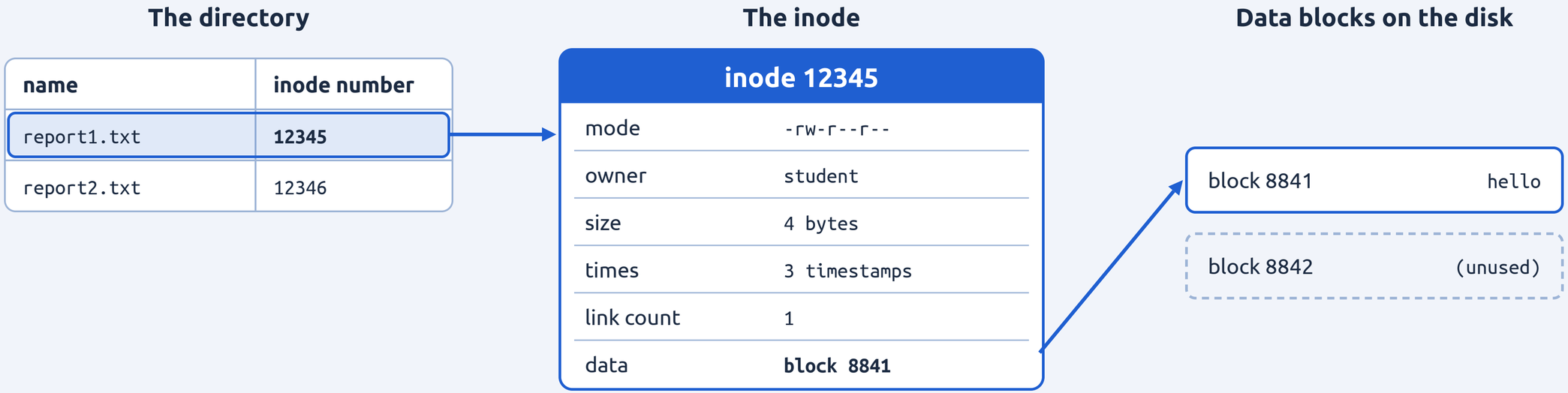
- The data of a file lives on the disk. An inode is the record that describes it: size, owner, permissions and where the data is.
- Every inode has a number. A file name is only an entry in a directory that points to an inode number.
- Two entries can point to the same inode. They are two names for one file: hard links.
- The data is freed only when the last name is gone.

inode number, permissions, link count, ...

```
$ ls -li
12345 -rw-r--r-- 2 student student 4 report1.txt
12345 -rw-r--r-- 2 student student 4 hard-demo.txt
99001 lrwxrwxrwx 1 student student 11 soft-demo.txt -> report1.txt
```

The 2 in the third column is the link count: two names point to inode 12345. The symbolic link has its own inode, 99001.

What an inode looks like



The directory maps a name to an inode number. The inode describes the file and says where its data is.

The name is not stored in the inode.

Read it from left to right: the name leads to an inode number, the inode leads to the data. ls -li shows that number.

Hard link and symbolic link

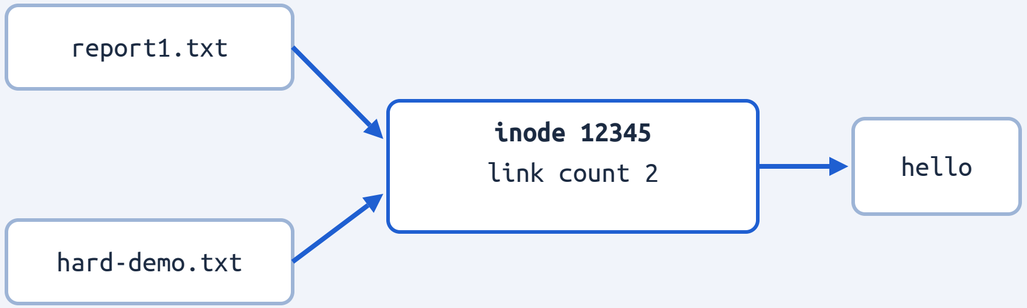
	Hard link	Symbolic link
Command	`ln target link`	`ln -s target link`
What it is	Another name for the same inode	A small file that stores a path
Inode	The same as the original	Its own inode
Original is removed	Data survives under the other name	The link breaks: it is dangling
Across filesystems	No	Yes
Folders	Not normally	Yes
Seen in ls -l	No arrow	An arrow to the stored target

Inspect with ls -li (inode numbers), stat (link count) and readlink (the stored path).

The two kinds of link, drawn

Hard link

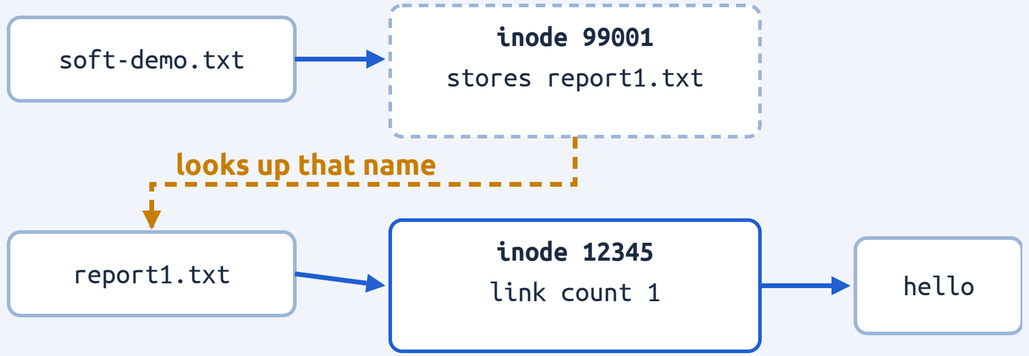
`ln report1.txt hard-demo.txt`



Two names for one inode.
Remove one name and the data stays.

Symbolic link

`ln -s report1.txt soft-demo.txt`



Its own inode, storing a path.
Rename the target and the path breaks.

A hard link is a second name for the inode. A symbolic link is a separate small file that holds a name to look up.

A relative target is read from the link's folder

student@server: ~/oslab-work/lab3

```
$ cd ~/oslab-work/lab3
$ ls -l links
lrwxrwxrwx 1 student student 9 maple-soft.txt -> maple.txt

$ cat links/maple-soft.txt
version 417
$ cd links && cat maple-soft.txt
version 417
```

The stored target maple.txt has no / at the front, so it is relative.

It is resolved from the folder that holds the link (links/), not from the folder where you are standing.

That is why the same link works from two places. Write the target as if you were standing next to the link.

A symbolic link repeats a name

example values, in links/

```
$ mv maple.txt maple-old.txt
$ cat maple-soft.txt
cat: maple-soft.txt: No such file or directory
$ printf 'version 463\n' > maple.txt
$ cat maple-soft.txt
version 463
$ cat maple-hard.txt
version 417
```

The link stores only the text maple.txt. After the rename no file has that name, so the link is dangling.

A new file with the old name makes it work again, but it now reads the new file. A hard link never does this: it follows the inode.

This is the surprise of Core 2. The hard link made before the rename still prints the original version.

The key experiment

student@server: ~/oslab-work/lab3

```
$ echo 'config data' > original.txt
$ ln original.txt hardlink.txt
$ ln -s original.txt symlink.txt
$ ls -li
$ rm original.txt
$ cat hardlink.txt
config data
$ cat symlink.txt
cat: symlink.txt: No such file or directory
```

echo with > writes the text into a new file. rm removes only the name original.txt.

The data still has one name, hardlink.txt, so it survives.

symlink.txt still says original.txt, and no such name exists. The link is dangling.

In practice: ls -la /usr/bin/python* shows programs that are managed with symbolic links.

LAB 3

Wildcards and links

A report is renamed. Which references still work, which break, and what does a new file with the old name do?

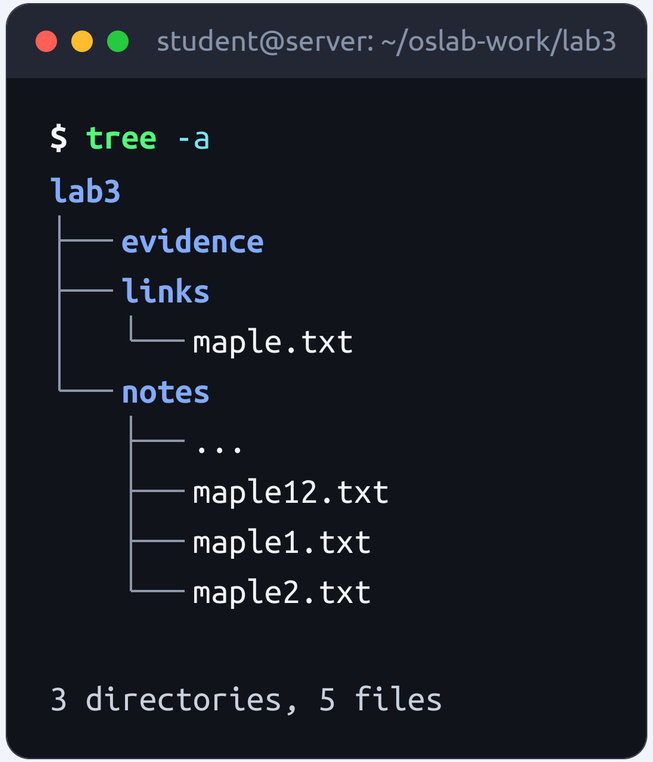
The scenario

SCENARIO

A report with many references

Rename a report, see which references break, repair the symbolic link, and meet a new file that takes the old name.

Your names and your version numbers are your own. This guide shows example values.



Inside the 120 minutes

Minutes	What happens
0 to 5	Setup
5 to 20	Guided: names, inodes and links
20 to 30	Prediction, then class discussion
30 to 40	Core 1: wildcards on your own files
40 to 60	Core 2: rename, break, and reuse the old name

Minutes	What happens
60 to 75	Core 3: repair and prove it
75 to 90	Plus and Challenge (or finish Core)
90 to 105	Live checkpoint
105 to 115	Debrief
115 to 120	Submit

Six helper commands, and why you use them

Command	What it does
`oslab values lab3`	Shows your report name, the word for the renamed file and your two version numbers. Use them everywhere
`oslab predict lab3`	Saves your prediction once, before the experiment
`oslab hint lab3 1`	A hint, levels 1 to 3. Hints cost no points
`oslab check lab3`	Looks at your files and shows your milestones
`oslab checkpoint lab3`	Opens when the instructor says so, near the end
`oslab debrief lab3`	After the lab: you give the right answers to your prediction questions

Before the lab, run oslab doctor and oslab prelab lab3. Read Concepts First as well. It takes about twenty minutes.

How you work

Core, Plus, Challenge

Core is for everyone. Plus and Challenge are for students who finish early. Full marks need only Core and the checkpoint.

AI tools

Allowed in Core, Plus and Challenge, so test what they tell you. Not allowed in the prediction and the checkpoint.

You write the repair

In Core 3 nobody gives you the repair commands. You decide the target and write both commands.

A wrong prediction costs nothing. It is marked for being made and corrected, not for being right. No step needs sudo.

Setup: start the lab

student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab3
$ cd "$OSLAB_WORKSPACE/lab3"
$ mkdir -p evidence
$ oslab values lab3
```

- The first line sets OSLAB_WORKSPACE to ~/oslab-work, unless it is already set. Type it as it is.
- oslab start lab3 makes the lab folder. It keeps your work if you run it again.
- cd takes you into it, and mkdir -p makes the evidence folder.
- oslab values lab3 prints your own names and numbers.

Your values: a name, a tag and two numbers

The last command prints them

example: your values will be different

```
$ oslab values lab3
Your personal values for lab3 (your-account):
  name     = maple
  tag      = old
  v1       = 417
  v2       = 463
```

What they are

- name is your report, for example maple.txt.
- tag is the word you add when you rename it: maple-old.txt.
- v1 is the version number inside your report. v2 is the number in the new file.
- Your values are different from the example, and from your neighbour's.

Type your values into shell variables

example values: use your own

```
$ name=maple
$ tag=old
$ v1=417
$ v2=463
```

- A variable is a name that holds a value. Write name=value with no spaces around the =.
- After this, the shell replaces "$name" with your report name, "$tag" with your word, and so on.
- Always write the variable in double quotes.
- If you open a new terminal, run cd and set the variables again.

Everything in the lab uses these four variables. Do not copy the example values. Use the ones oslab values lab3 printed.

Angle brackets stand for your own values

The instruction says	With the example values, you type	Or, with the variables
`ln "$name.txt" "$name-hard.txt"`	`ln maple.txt maple-hard.txt`	the same line
`<name>-<tag>.txt`	`maple-old.txt`	`"$name-$tag.txt"`
`version <v2>`	`version 463`	`printf 'version %s\n' "$v2"`

Never type the angle brackets. Type your own value, or the variable in double quotes. The - in "$name-hard.txt" ends the variable name, so the shell reads $name and then the text -hard.txt.

Your folder

student@server: ~/oslab-work/lab3

```
$ tree -a -L 3
lab3
├── evidence
├── links
│   └── maple.txt
└── notes
    ├── maple12.txt
    ├── maple1.txt
    ├── maple2.txt
    ├── maple.log
    └── .maple.txt

3 directories, 6 files
```

links/ holds your report, with your name. It contains version 417 in the example.

notes/ holds five files with your name: two with one digit, one with two digits, a log and a hidden file.

oslab check lab3 shows TRY on every line now. The hidden .oslab-managed.json is the helper's. Leave it.

GUIDED

Names, inodes and links

Five to twenty minutes, with the instructor. A separate practice folder, so your own files stay untouched.

Step 1: three clearly different names

student@server: ~/oslab-work/lab3

```
$ mkdir -p practice
$ printf 'one\n' > practice/report1.txt
$ printf 'two\n' > practice/report2.txt
$ printf 'log\n' > practice/report.log
$ ls practice
report1.txt  report2.txt  report.log
```

This part uses fixed names in practice/. Your own names come back in Core 1.

printf with > writes one line into a new file. The names differ on purpose: two .txt files and one .log file.

Step 1: how the shell expands a pattern

student@server: ~/oslab-work/lab3

```
$ printf '<%s>\n' practice/*.txt
<practice/report1.txt>
<practice/report2.txt>
$ printf '<%s>\n' practice/report?.txt
<practice/report1.txt>
<practice/report2.txt>
$ printf '<%s>\n' 'practice/*.txt'
<practice/*.txt>
```

Bash expands * and ? before printf runs. printf prints each name it receives between < and >.

The quoted pattern is not expanded, so printf receives the literal text.

TRY

Observe

Why is report.log never in the list?

Step 2: your first two links

student@server: ~/oslab-work/lab3

```
$ ln practice/report1.txt practice/hard-demo.txt
$ ln -s report1.txt practice/soft-demo.txt
$ ls -li practice/report1.txt practice/hard-demo.txt practice/soft-demo.txt
$ readlink practice/soft-demo.txt
```

report1.txt and hard-demo.txt show the same inode number and the link count 2: one file, two names. The symbolic link has its own number.

readlink prints the text the link stores.

TRY

Think

soft-demo.txt is inside practice/ and stores only report1.txt. Which path would be wrong, and what would Linux look for?

PREDICT

Guess first, then try

Before you rename anything. Ten minutes.

The prediction

student@server: ~/oslab-work/lab3

```
$ oslab predict lab3
```

It asks three questions and one sentence, with your own names and numbers. Answer alone, without AI.

PREDICTION

The sequence

You rename your report. Then a new file takes the old name. What do the two links print?

HOW

The rules

The first answer counts. A wrong guess costs nothing.

Use what you know from Concepts First: what does each kind of link store?

Wildcards on your own files

Ten minutes, from minute 30 to 40.

Step 1: look at your notes folder

example values: your names are different

```
$ cd "$OSLAB_WORKSPACE/lab3"
$ ls -a notes
.  ..  .maple.txt  maple1.txt  maple12.txt  maple2.txt  maple.log
```

ls -a also shows hidden names: the ones that start with a dot.

TRY

Think

Which of these files will the pattern notes/*.txt select? Which will it skip, and why?

Step 2: three patterns, and the middle one is yours

student@server: ~/oslab-work/lab3

```
$ {
$   printf '<%s>\n' notes/*.txt
$   # write the middle command here
$   printf '<%s>\n' 'notes/*.txt'
$ } | tee evidence/wildcards.txt
```

YOUR TURN

You write the middle command

It must print only your two files with a one-digit number, maple1.txt and maple2.txt, and not maple12.txt. Write the variable in double quotes and the ? outside them.

{ and } group the three commands, so one tee saves all their output.

What you should see

Command	Prints, with the example values
`notes/*.txt`	maple1.txt, maple12.txt and maple2.txt. Not the hidden file, and not maple.log
`your middle command`	maple1.txt and maple2.txt only
`'notes/*.txt'`	The literal text: <notes/*.txt>

The most common mistake is a * where the ? belongs: then maple12.txt is in the list too. oslab hint lab3 1 helps.

CORE 2

Rename, break, and reuse the old name

Twenty minutes, from minute 40 to 60. Keep your own names: oslab check looks for them.

Step 1: make the references, and save the starting state

student@server: ~/oslab-work/lab3/links

```
$ cd "$OSLAB_WORKSPACE/lab3/links"
$ cat "$name.txt"
$ ln "$name.txt" "$name-hard.txt"
$ ln -s "$name.txt" "$name-soft.txt"
$ {
$   printf '\n=== BEFORE RENAME ===\n'
$   ls -li "$name.txt" "$name-hard.txt" "$name-soft.txt"
$   printf 'hard: '; cat "$name-hard.txt"
$   printf 'soft: '; cat "$name-soft.txt"
$   printf 'stored target: '; readlink "$name-soft.txt"
$ } | tee ../evidence/links.txt
```

The two links are made before the rename. tee starts evidence/links.txt, and the labelled line marks the state.

You should see version 417 three times, and the same inode number for your report and the hard link.

CAREFUL

If ln says File exists

Look at the existing entry first. Do not overwrite work blindly.

Step 2: rename the report, then look

student@server: ~/oslab-work/lab3/links

```
$ mv -- "$name.txt" "$name-$tag.txt"
$ {
$   printf '\n=== AFTER RENAME, BEFORE REPAIR ===\n'
$   ls -li "$name-$tag.txt" "$name-hard.txt" "$name-soft.txt"
$   printf 'hard: '; cat "$name-hard.txt"
$   printf 'stored target: '; readlink "$name-soft.txt"
$   printf 'soft: '; cat "$name-soft.txt"
$ } 2>&1 | tee -a ../evidence/links.txt
```

mv -- renames the entry; the file keeps its inode. tee -a adds to the file. 2>&1 puts the error message into the evidence.

THINK

The error is expected

cat of the symbolic link fails with No such file or directory. The link exists, but the name it stores belongs to no file now: it is dangling.

Step 3: a new file takes the old name

student@server: ~/oslab-work/lab3/links

```
$ printf 'version %s\n' "$v2" > "$name.txt"
$ {
$   printf '\n=== NEW FILE USING THE OLD NAME ===\n'
$   ls -li "$name.txt" "$name-$tag.txt" "$name-hard.txt" "$name-soft.txt"
$   printf '%s: ' "$name.txt"; cat "$name.txt"
$   printf 'hard: '; cat "$name-hard.txt"
$   printf 'soft: '; cat "$name-soft.txt"
$ } 2>&1 | tee ../evidence/replacement.txt
```

Somebody else makes a new report with your old file name. It holds version 463 in the example.

SURPRISE

Look closely

The symbolic link works again, but it prints the NEW number. The hard link still prints your original number: it follows the inode, not the name.

What each name prints in each state

State	Hard link	Symbolic link
Before the rename	version 417	version 417
After the rename	version 417	cat fails: dangling
After the new file	version 417	version 463, the wrong file

Example values. Write in your report which of the two was a surprise. The hard link and the renamed report keep one inode all the time.

CORE 3

Repair and prove it

Fifteen minutes, from minute 60 to 75.

Lab 3

Step 1: repair only the symbolic link

What to do

1. Make maple-soft.txt read your original report again, which is now maple-old.txt.
2. Use a relative target. Ask: from which folder is the stored target read?
3. Do not touch the hard link. Do not rename the report back.
4. Check: cat "$name-soft.txt" must print version 417.

YOUR TURN

Your decision, your commands

Delete the link with rm --, then make it again with ln -s TARGET LINK. Nobody gives you TARGET.

student@server: ~/oslab-work/lab3

```
$ rm -- LINK
$ ln -s TARGET LINK
$ cat "$name-soft.txt"
```

Step 2: save the repaired state, and your decision

student@server: ~/oslab-work/lab3/links

```
$ {
$   printf '\n=== AFTER REPAIR ===\n'
$   ls -li "$name-$tag.txt" "$name-hard.txt" "$name-soft.txt"
$   printf 'stored target: '; readlink "$name-soft.txt"
$   printf 'hard: '; cat "$name-hard.txt"
$   printf 'soft: '; cat "$name-soft.txt"
$ } | tee -a ../evidence/links.txt
$ nano ../evidence/links.txt
```

In nano, add one line at the end. It starts with Decision: and says why you stored a relative target.

Say also what would break if the link stored an absolute path and you moved the whole folder. Save with Ctrl+O and Enter, leave with Ctrl+X.

Step 3: check your milestones

example values

```
$ cd "$OSLAB_WORKSPACE/lab3"
$ oslab check lab3
PASS: wildcards: evidence/wildcards.txt holds your three expansions
PASS: hard link: maple-hard.txt and maple-old.txt are the same file (version 417)
PASS: symbolic link: maple-soft.txt stores a relative target that reaches maple-old.txt
PASS: new file: maple.txt holds version 463 and is separate from the version 417 file
PASS: evidence/links.txt records the failure after the rename and the repaired link
milestones: 5/5
```

A TRY line names what is missing, and the why: lines say what oslab found instead. Fix it and check again.

Core is complete when oslab check lab3 shows milestones: 5/5. Hints: oslab hint lab3 1, 2, 3.

Plus: one more pattern

[12] matches one character, 1 or 2. [!1] matches one character that is not 1.

Write down which names you expect before you run them. Then save one sentence in evidence/plus.txt: where did your guess differ?

student@server: ~/oslab-work/lab3

```
$ cd "$OSLAB_WORKSPACE/lab3"
$ printf '<%s>\n' notes/"$name"[12].txt
$ printf '<%s>\n' notes/"$name"[!1]*
```

PLUS

Hint

The second pattern needs one character that is not 1, then anything.

Plus and Challenge are not needed for full marks. They are noted in your feedback.

Challenge: how many names does a file have?

student@server: ~/oslab-work/lab3

```
$ mkdir -p extra
$ cd extra
$ printf 'keep me\n' > one.txt
$ ln one.txt two.txt
$ ln one.txt three.txt
$ ls -l one.txt two.txt three.txt
```

The number after the permissions is the link count. Then rm -- one.txt, look again, and cat two.txt.

CHALLENGE

Answer in evidence/challenge.txt

What link count do the remaining names show after each rm? Is the data gone after you delete one.txt? When would it be gone?

LIVE CHECKPOINT

Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

Why is there a checkpoint?

It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them.

The names are new

The questions use a new folder and new file names, made for you at that moment. An answer copied from a neighbour does not fit.

The instructor sees who needs help

Your answers arrive at once, so a common mistake can be explained before the next lab.

It is fair

Everyone gets the same kind of task, with their own names.

What happens, step by step

1. The instructor opens it
Close your AI tools

→

2. oslab checkpoint lab3
Answer the five questions

→

3. Build and repair
A chain of two links; you repair it

→

4. oslab check lab3
A sixth milestone must pass

Before it opens

The command says that the checkpoint is not open yet. Wait for the instructor.

While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

When you finish

oslab check lab3 shows whether the sixth milestone, checkpoint, passes. You can fix the repair and check again.

What the checkpoint asks

Part	What you do
Five questions	They use a new folder, new names and a chain of two symbolic links. Four are marked; the last is one sentence that your instructor reads.
The case	Build the case with the commands that oslab prints, repair first.txt yourself, and save the output in evidence/checkpoint.txt.
The check	Run oslab check lab3. The sixth milestone, checkpoint, must show PASS.

The same idea as Core 3: a link stores a name, and a rename breaks it. Here a second link points at the first.

Look back, then hand in

Ten minutes of debrief, five to submit.

Debrief: look at your prediction again

student@server: ~/oslab-work/lab3

```
$ cat "$OSLAB_WORKSPACE/.records/lab3-predict.json"
$ oslab debrief lab3
```

oslab saved your prediction in a hidden folder, .records. It stays as it was, so the comparison shows which answers you confirmed and which you corrected.

DISCUSS

Class discussion

Why can a symbolic link work and still read the wrong file?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab3/evidence"
$ cd "$OSLAB_WORKSPACE/lab3"
$ cp -- evidence/wildcards.txt evidence/links.txt "$SUBMISSION_REPO/lab3/evidence/"
$ cp -- evidence/replacement.txt "$SUBMISSION_REPO/lab3/evidence/"
$ cp -- evidence/checkpoint.txt "$SUBMISSION_REPO/lab3/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

If you did Plus or Challenge, copy evidence/plus.txt or evidence/challenge.txt the same way. The course website checks that README.md, wildcards.txt, links.txt and replacement.txt are there, with these exact names.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab3
├── evidence
│   ├── checkpoint.txt
│   ├── links.txt
│   ├── replacement.txt
│   └── wildcards.txt
└── README.md

1 directory, 5 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

Submit 3: commit and push

student@server: ~

```
$ cd ~/os-gic-YOUR_ID
$ git pull
$ git add os-lab-YOUR_ID/lab3
$ git commit -m "Lab 3"
$ git push
```

git pull comes first, in case you pushed from your own computer since.

Replace YOUR_ID with your own ID, as before.

How it is marked: 10 points

Evidence for the lab objectives	Points
Correct files: oslab check lab3 passes Core milestones 1 to 4	2
Tests: the failure after the rename and the repaired link in links.txt (milestone 5)	2
Prediction saved in time, and corrected with oslab debrief lab3	2
Live checkpoint: the four answers, the repaired chain and the saved output, and your sentence	3
Complete submission: the required files in your repository, on time	1

Every row is marked from your oslab records and the files in your repository, so run oslab check lab3 until it shows what you expect. Your prediction earns its first point for being made, not for being right. Plus and Challenge are not needed for full marks.

Common mistakes

Mistake	What you see	Fix
A space around the = in name=maple	An error, or an empty variable	Write it with no spaces
Quotes around the whole pattern	The literal pattern is printed	Quote only the variable: "$name"?.txt
The hard link made after the rename	It names the new file, so the experiment proves nothing	Make both links before mv
The repair uses the old name	The link reads the new file, version 463	Point it at the renamed report, maple-old.txt
An absolute target in the repair	oslab check says it must be relative	Write the target as if you stood next to the link
Predicting after the rename	No prediction saved in time	Run oslab predict lab3 before Core 1

GRUB and shared libraries

Optional extensions. They need a disposable virtual machine or a compiler, and they are not part of the two-hour core.

From power-on to login

Stage	What it does
BIOS or UEFI	Self-test, then finds the boot device
GRUB	Shows a menu and loads the kernel
Kernel	Starts the hardware, mounts the root filesystem, runs the first process (PID 1)
init or systemd	Starts the system services in order
Login	User space is ready

GRUB files and key settings

File	Purpose	Edit?
`/boot/grub/grub.cfg`	The generated menu	Never
`/etc/default/grub`	Timeout, kernel parameters	Yes
`/etc/grub.d/`	Scripts, custom entries	Yes
`/boot/vmlinuz-*`	The compressed kernel	No

Setting	Meaning	Example
`GRUB_DEFAULT`	Entry to boot	0
`GRUB_TIMEOUT`	Seconds at the menu	10
`GRUB_CMDLINE_LINUX_DEFAULT`	Kernel parameters	quiet splash
`GRUB_BACKGROUND`	A menu background image	/boot/grub/bg.png

After you edit /etc/default/grub, run sudo update-grub. Custom entries go in /etc/grub.d/40_custom. Do this only in a disposable virtual machine, never on the shared server.

GRUB recovery

Mode	How you enter	Use
Recovery mode	GRUB menu, Advanced options, recovery	A root shell, fsck, limited services
GRUB command line	Press c at the menu	Boot by hand when the menu is broken
Live USB	Boot from an external image	Full rescue when GRUB is gone

grub>

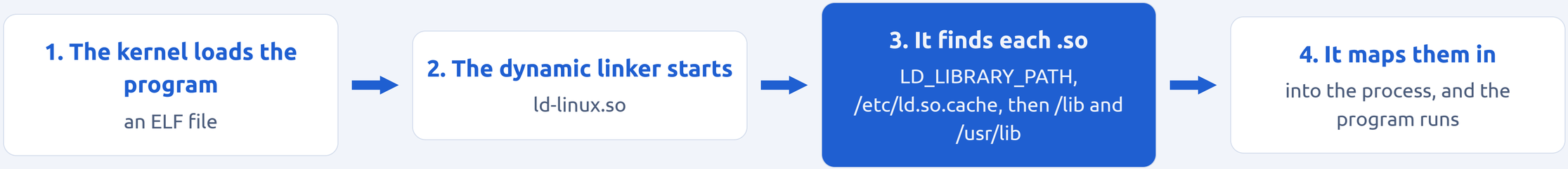
```
$ set root=(hd0,msdos1)
$ linux /boot/vmlinuz-VERSION root=/dev/sda1
$ initrd /boot/initrd.img-VERSION
$ boot
```

Take a snapshot of the virtual machine before you break anything on purpose.

Static and dynamic linking

	Static	Dynamic
When linked	At compile time	When the program starts
Library code	Copied into the executable	Loaded from .so files
Executable size	Larger	Smaller
Memory	Each program has its own copy	Programs share one copy
Updates	Recompile every program	Replace the .so, and all programs benefit
File extension	.a (archive)	.so (shared object)

How a program finds its libraries



Looking at shared libraries, and building one

student@server: ~/oslab-work/lab3

```
$ ldd /bin/ls
$ ldconfig -p | grep libc
$ gcc -fPIC -c mylib.c -o mylib.o
$ gcc -shared -o libmylib.so mylib.o
$ gcc main.c -L. -lmylib -o myprogram
$ LD_LIBRARY_PATH=. ./myprogram
```

ldd lists the libraries a program needs. ldconfig -p searches the library cache.

-fPIC makes position-independent code, so the library can load at any address. -shared makes the .so file.

For a user-local library, set LD_LIBRARY_PATH instead of installing into system folders.

More ways to look at a library

student@server: ~/oslab-work/lab3

```
$ file /lib/x86_64-linux-gnu/libc.so.6
$ readelf -h /bin/ls
$ grep '[.]so' /proc/$$/maps
$ sudo ldconfig
```

file tells you the kind of file: an ELF shared object.

readelf -h prints the ELF header of a program. /proc/$$/maps lists what the current shell has loaded.

After you install a new .so in a system folder, sudo ldconfig refreshes the cache. Not on the shared server.

Key takeaways

01

Wildcards

The shell expands a pattern into names before the command runs. Quote a pattern to keep it literal.

02

Name and inode

A file name is an entry that points to an inode. Hard links are more names for one inode.

03

Symbolic links

A symbolic link stores a path. A relative target is read from the link's folder.

04

Evidence

Rename, look, repair, replace. Each step has a record you can check.

Next session

COMING UP

Lab 4: I/O redirection, pipelines and processes

Send output to files, connect commands with pipes, and see and stop processes.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapters 13 and 14, File-System Interface and File-System Implementation.